Computers & Security 154 (2025) 104447

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Vul_{diff}Finder: Discovering inconsistencies in unstructured vulnerability information

Qindong Li [a], Wenyi Tang [a,b,c,*], Xingshu Chen [a,c], Hao Ren [a,c]

[a] School of Cyber Science and Engineering, Sichuan University, Chengdu, 610065, China
[b] Intelligent Policing and National Security Risk Management Laboratory, Luzhou, 646000, China
[c] Key Laboratory of Data Protection and Intelligent Management (Sichuan University) Organization:Ministry of Education, Chengdu, 610065, China

ARTICLE INFO

Keywords:
Vulnerability information
Information quality
Semantic inconsistency
Syntactic hierarchy
SpanBERT

ABSTRACT

The information conveyed by vulnerability reports is crucial for enhancing the security of information systems. Nonetheless, there are widespread information inconsistencies across reports, including, numerical discrepancies, misreported version ranges, semantic conflict, and so on. Identifying these inconsistencies is essential for improving information quality. Current research primarily focuses on standardized, non-free-form information's inconsistency at the character or numerical level, while research for unstructured ones at the semantic level is limited. Given this, we introduce Vul_{diff}Finder to determine the inconsistency of unstructured vulnerability information at the semantic level. Firstly, it utilizes NLP tools to break down unstructured information into constituent sets, and design a determination strategy based on the constituent's syntactic hierarchies and semantic similarity. The designed strategy can determine information pairs in arbitrary structure. Secondly, it creates a span similarity-based fine-tuning task to enhance the embedding capabilities of the SpanBERT model, ensuring accurately capturing semantic information in the vulnerability domain. Finally, a dataset containing eight categories of vulnerability information and 1,612 samples is utilized to validate the proposed method. The results demonstrate that Vul_{diff}Finder outperforms the state-of-the-art schemes, showing a 4.31% improvement in the F1-score. Additionally, we discover that consistency is higher in information that has simpler writing structures (up to 56.46%). Heterogeneous and Contained are often found in information with fixed or complex writing structures (up to 23.33% and 38.30%, respectively). Divergent and Repugnant mainly occur in information with a high missing rate.

Contents

1. Introduction 2
2. Background and challenges 3
   2.1. Vulnerability aspects 3
   2.2. Inconsistency situations 3
   2.3. Challenges 4
      2.3.1. Challenge 1. Determination strategy covers arbitrary writing structures 4
      2.3.2. Challenge 2. Capture the semantics of vulnerability domain knowledge accurately 4
3. Methodology 4
   3.1. System workflow overview 4
   3.2. Aspect decomposer 5
      3.2.1. Writing structure categories 5
      3.2.2. Decomposition principles 5
   3.3. Inconsistency determinator 5
      3.3.1. Syntactic hierarchical tree construction 6
      3.3.2. Level-by-level inconsistency determination and integration 6
   3.4. Language model fine-tuning 6
4. Argumentation 7

* Correspondence to: Sichuan University, Chengdu 610065, China.
E-mail addresses: liqindong@stu.scu.edu.cn (Q. Li), wtang@scu.edu.cn (W. Tang), chenxsh@scu.edu.cn (X. Chen), hao.ren@scu.edu.cn (H. Ren).

https://doi.org/10.1016/j.cose.2025.104447
Received 26 August 2024; Received in revised form 22 January 2025; Accepted 14 March 2025
Available online 29 March 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

4.1. Datasets ... 7
4.1.1. Domain-specific span-similarity dataset ... 7
4.1.2. Aspect-inconsistency dataset ... 7
4.2. Experimental schemes ... 7
4.2.1. Ablation experiments ... 8
4.2.2. Baseline models ... 8
4.2.3. Evaluation metrics ... 8
4.3. Experiment results ... 8
4.3.1. Effectiveness of aspect decomposer ... 8
4.3.2. Effectiveness of LM fine-tuning ... 8
4.3.3. Effectiveness of inconsistency determinator ... 9
4.3.4. Compared with other works ... 10
4.4. Result analysis ... 10
5. Discussions ... 12
5.1. Implications ... 12
5.2. Future work ... 12
6. Related work ... 12
6.1. Applications of vulnerability report ... 12
6.2. Vulnerability information inconsistency analysis ... 12
7. Conclusion ... 12
CRediT authorship contribution statement ... 12
Declaration of competing interest ... 12
Acknowledgments ... 13
Data availability ... 13
References ... 13

1. Introduction

Open-source vulnerability reports are an important channel for security staff to obtain vulnerability dynamics, plays an important role in securing information systems. The information contained in the reports (e.g. version, attack vectors, impact, etc.) is important for downstream tasks, such as vulnerability detection, patch development, and cyber risk assessment Cheng et al. (2023), Fan et al. (2020).

Currently, some studies have shown several problems within vulnerability reports, including missing necessary information Chaparro et al. (2017a), containing incorrect information Nappa et al. (2015), Nguyen and Massacci (2013), and information inconsistent across records Jiang et al. (2021). Cross-Record Inconsistency(CRI) indicates scenarios where the same attributes of vulnerability are conflicting, or where multiple attributes indicate contradicting vulnerability characteristics. For example, the affected software versions reported by NVD and Gentoo for CVE-2008-5036 are "<0.9.8a" and "0.9.x before 0.9.6", respectively, which is a typical CRI because both reported inconsistent version ranges. This inconsistency raises concerns about the quality of vulnerability information for intelligence consumers and significantly hampers its adoption Chaparro et al. (2017b), Mu et al. (2018). Comprehensive and accurate discovery of these inconsistencies is the cornerstone of reversing this situation

Information inconsistency measurement has attracted significant attention. Nappa Nappa et al. (2015), Dong Dong et al. (2019), and Cheng Cheng et al. (2023) analyzed the differences in the range of affected software versions among mainstream vulnerability databases, and found that overclaims and underclaims were prevalent. Sun Sun et al. (2023a) shows that character-level and semantic-level differences in software names are common among mainstream vulnerabilities. Farhang Farhang et al. (2020), on the other hand, focuses on the differences in the time dimension of vulnerability information released in security bulletins among vendors. Jiang Jiang et al. (2021) found inconsistencies in the CVSS scores, vulnerability types, and other dimensions between databases such as NVD and real environment vulnerability determination results. From the perspective of the information types oriented to the existing research, it is mainly devoted to that with the equivalent format and simple structure, such as disclosure time, CVSS score, software name, etc. There is a lack of research oriented towards inconsistency determination among unstructured information with complex structure and flexible writing. In terms of the determination strategies, existing studies mainly adopt rigid strategies to achieve character-level determination, such as spelling errors and inconsistent quantitative information. There also lack of studies on semantic-level consistency determination.

Unstructured information, represented by impact and attack vector, is widely used to support tasks such as threat hunting and attack tracing. There is currently an urgent need for work on determining the inconsistency of this type of information. Therefore, the research objective of this paper is to measure the semantic inconsistency of unstructured vulnerability information, which face two main challenges:

(1) The determination strategy covers arbitrary writing structures. As shown in Table 1, the two impacts (blue) exist with structures of "verb+object" and "verb+object+complement" respectively, the strategy should cover the determination of various types of unstructured information.

(2) Accurately capturing the semantics of domain knowledge. In Table 1, the written structures of version (red) in reports A and B are different while with the same semantics, the language model should accurately capture the semantics of this non-standard form text.

In this paper, we introduce the **Vul**nerability **diff**erences **Finder** (Vul_{diff}Finder) to overcome the above challenges. For the first challenge, we uniformly split the text of arbitrary writing structure into "atomic units" (comprised solely of words and noun phrases) via POS and syntactic parsing. Subsequently, we design a determination strategy based on syntactic hierarchical between "atomic units". It utilizes the Siamese Network to determine the semantic similarity between texts of the same hierarchy. For the second challenge, we introduce a POS-weighted embedding method to fine-tune the SpanBERT model. It strengthens the ability to embed vulnerability domain knowledge.

We collected vulnerability reports from 9 sites (including: NVD, IBM, Exploit-DB, Packet Storm, VULHUB, etc.) and extracted 8 dimensions of vulnerability information using the Vul-PAG model Li et al. (2024). Then, a dataset containing 1612 samples with 5 inconsistency categories is constructed through scientific annotation, on which the proposed approach is validated and compared with other works. Experimental results show that, Vul_{diff}Finder outperformed the state-of-the-art schemes and achieving precision, recall, and F1-score exceeding 5.20%, 3.87%, and 4.52%, correspondingly. It also reveals some interesting findings. "Consistent" exhibits the highest proportion

Table 1
Two reports CVE-2020-28188.

Report	Part content of report
A	Remote Command Execution (RCE) vulnerability in TerraMaster TOS <= 4.2.06 allow remote unauthenticated attackers to inject OS commands via /include/makecvs.php in Event parameter.
B	... vulnerability in TerraMaster TOS versions 4.2.06 and below via shell metacharacters in the Event parameter at vulnerable endpoint include/makecvs.php during CSV creation. Any unauthenticated user can therefore execute commands on the system under the same privileges as the web application, which typically ....

in almost all aspects (up to 56.46%), indicates that there is a strong circulation of disclosure information among sites. "Heterogeneous" primarily occurs in the aspects of "version" and "root cause". Further scrutiny indicates that the former is influenced by synonymous terms such as "-", "to", "through", etc., whereas the latter delineates analogous behaviors through varied expressions. "Contained" predominantly appears in the aspects of "product", "component", "version " and "impact" (up to 38.30%), the first two due to the fact that some sites give "vendor" or "more complete file paths", while the latter two are due to the disclosure of more comprehensive information by some sites. "Divergent" primarily manifests in aspects such as "vulnerability type" (32.93%), "root cause" (27.14%), "attacker vector", and "impact" that indicating diverse observable behavioral patterns. This discrepancy likely stems from variations in validation methods and thematic focus among websites. "Repugnant" exhibits the lowest frequency across all aspects, primarily appearing in "component" (8.83%) and "attacker type". This observation underscores a critical challenge in information quality, as it is difficult for consumers to make binary choices.

Overall, our contribution comprises as following:

(1) The Vul_{diff}Finder is proposed to determine inconsistencies in writing structure and semantics of the identical vulnerability aspects among reports. To the best of our knowledge, it is the first time to comprehensively determine information inconsistencies in unstructured vulnerability aspects.

(2) A POS tagging and syntactic analysis-based simplification scheme is developed to address arbitrary text with complex writing structure.

(3) A hierarchical determination strategy based on the syntactic tree and the Siamese Network is devised to solve the scenario of difficulty in comprehensively covering the flexible writing structure among text pairs when determining.

(4) The constructed comprehensive, fine-grained dataset on inconsistency concerning IoT-related vulnerabilities serves as valuable resources for future research.[1]

This paper is organized as follows: In Section 2, we provide a more detailed description of the background and challenges. Section 3 outlines our approach in detail. In Section 4, we introduce the datasets used in this study and present our evaluation. Section 5 includes a discussion of the findings and limitations. Finally, Section 6, VII, and VIII cover Related Work, Future Work, and the Conclusion, respectively.

2. Background and challenges

Vulnerability reports have become a critical tool to help organizations identify and remediate security issues. However, vulnerability reports from different sources (e.g. vendor bulletins, third-party platforms, and researcher reports) often contain inconsistent information due to differences in perspective, terminology, and level of detail. Such inconsistencies can affect various aspects such as vulnerability description, scope of impact, severity rating, or remediation recommendations. Inconsistent information not only increases the complexity of vulnerability management but also leads to incorrect prioritization decisions that can delay the remediation process or introduce potential security risks. Therefore, accurately identifying and resolving information inconsistencies in multi-source vulnerability reports is critical to improving vulnerability management efficiency and ensuring system security. With this in mind, this section introduces the categories of inconsistent information, the types of inconsistencies, and the technical challenges in determining these inconsistencies.

2.1. Vulnerability aspects

The information contained in the vulnerability report describes the vulnerability from multiple perspectives, and the different perspectives are also referred to as vulnerability aspects. MITRE recommends using nine aspects to describe vulnerability based on the following two templates[2]:

(1) [*Vulnerability Type*] in [*Component*] in [*Vendor*] [*Product*] [*Version*] allows [*Attacker type*] to [*Impact*] via [*Attack Vector*];

(2) [*Component*] in [*Vendor*] [*Product*] [*Version*] [*Root Cause*], which allows [*Attacker type*] to [*Impact*] via [*Attack Vector*].

For convenience, we use the abbreviations in parentheses to represent different aspects: Vulnerability Type (VT), Component (VP), Product (VN), Version (VV), Attacker Type (VAT), Impact (VR), Attacker Vector (VAV), and Root Cause (VRC).

2.2. Inconsistency situations

Information inconsistency among multi-source vulnerability reports is a common issue. Jiang Jiang et al. (2021) classify this inconsistency into two categories: Record-level inconsistency and Cross-record inconsistency. In terms of specific inconsistency scenarios, Dong Dong et al. (2019) first defined these as Overclaim and Underclaim, while Sun Sun et al. (2023c) further expanded on this by introducing complete mismatching, overlapping, and exact matching. These categories primarily focus on standard non-free-form fields and their quantifiable comparison levels, which do not address the semantic inconsistencies found in unstructured information. We observe that inconsistencies in unstructured vulnerability information often appear in terms of writing structure and semantics. These include issues such as writing heterogeneity, semantic containment, semantic repugnance, and semantic divergence. Based on the work of Sun Sun et al. (2023c), we have defined five categories that effectively measure the consistency between unstructured vulnerability messages. For a pair of information:

Consistent: if the entities within two aspects are the same in number, semantic, and writing structure, we consider them as Consistent. As shown the *VP* of NVD and Gentoo in Fig. 1.

Heterogeneous: if the entities of "main" and "attachment" are the same in number and semantics while exhibiting substantial writing differences, we consider them as Heterogeneous. As shown the *VN* in IBM and Gentoo signifies the same affected software while showing significant writing differences.

Contained: if the entities of "main" are inclusive in number and semantic, or the "main" is the same while the "attachment" is inclusive,

[1] CVE aspects inconsistency dataset: https://github.com/dong-xs/intelligence_confict_dataset.

[2] Vulnerability Description Templates: https://cveproject.github.io/docs/content/key-details-phrasing.pdf.

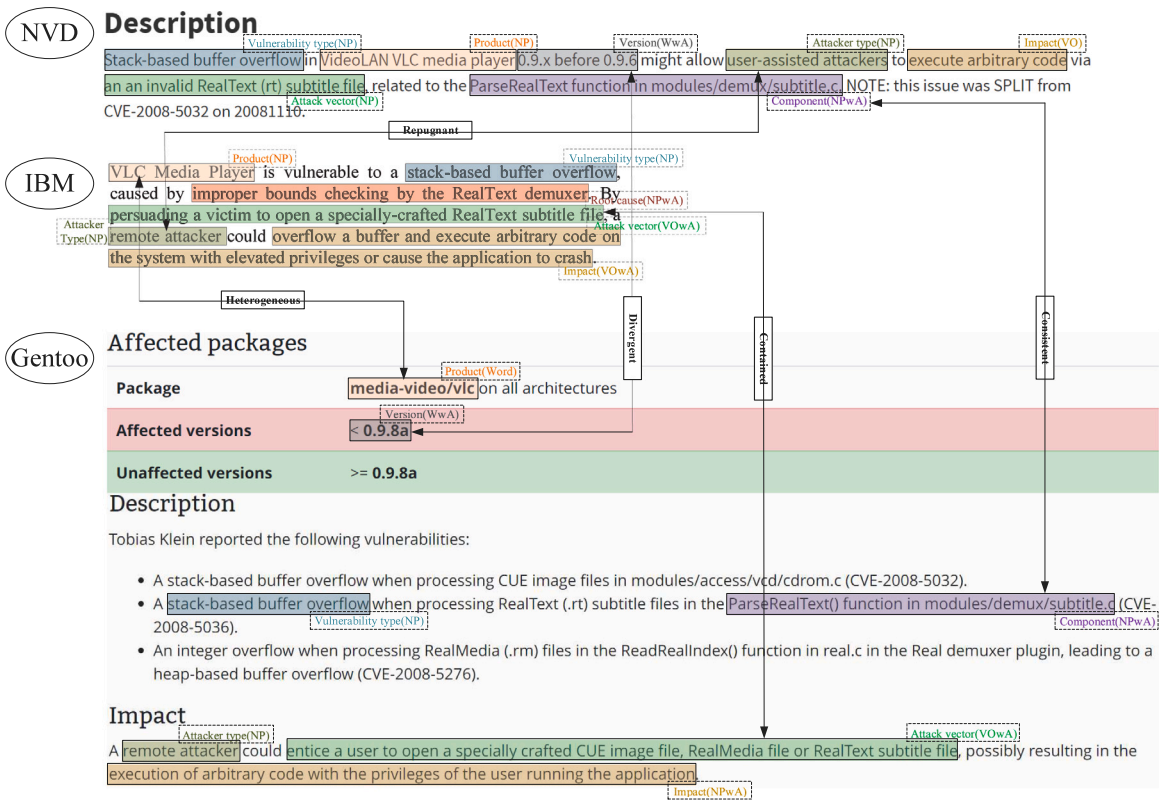


Fig. 1. The illustration is contextualized within CVE-2008-5036 vulnerability reports sourced from NVD, IBM, and Gentoo. Extracted aspects are color-coded, and their writing structures are identified as Word, NP, OVwA, etc. Furthermore, the consistency of identical aspects across reports is noted as Consistent, Contained, Heterogeneous, etc.

we consider them as Contained. For example, the *VAV* in IBM and Gentoo, the entities' number and semantic in the "main" are consistent, both "attachment" contains an entity of "RealText subtitle file", while the latter contains entities of "CUE image file" and "RealMedia file" additionally, which can be categorized into Contained.

Divergent: if the entities within the "main" exhibit complete semantic inconsistency, we consider them as Divergent. For instance, the *VV* in NVD and Gentoo demonstrates obvious semantic disparities, which is a typical content conflict Dong et al. (2019).

Repugnant: if the "main" contains the same semantic entity, but their modifiers differ semantically. Alternatively, two "main" entities are identical, and their respective "attachment" contains the same entity with differing modifiers, we consider them as Repugnant. For instance, the *VAT* in NVD and IBM label "attacker" as "main", and respectively use "user-assisted" and "remote" to modify them, illustrating distinct attacker type.

2.3. Challenges

Our goal is to measure the semantic inconsistency between arbitrary unstructured vulnerability information pairs. However, we mainly faced the following two technical challenges:

2.3.1. Challenge 1. Determination strategy covers arbitrary writing structures

A vulnerability information can have different writing structures, e.g., "version" can be a word or phrase (e.g., "1.20B10_BETA" and "prior to 4.34"); "impact" can be a word, a phrase, or a verb-object complex (e.g., "DDoS", "execute code","gain access to sensitive information"). It is difficult for a determination strategy to cover information pairs between various writing structures simultaneously and determine their inconsistency, e.g. word vs. phrase, phrase vs. verb-object compound.

2.3.2. Challenge 2. Capture the semantics of vulnerability domain knowledge accurately

Vulnerability reports include numerous proper nouns and out-of-vocabulary (OOV) words, rendering it challenging for language models to accuracy embedding. Additionally, aspects exist not only in the form of words but also spans, simultaneous embedding texts with significantly different lengths further elevates the difficulty of the language model. These factors contribute to the obstacles of capturing the nuanced distinctions within vulnerability knowledge.

3. Methodology

Fig. 2 depicts the architecture of Vul_{diff}Finder. It takes raw reports and extracted aspects as input and comprises three primary components: **Aspect Decomposer(AD)**, **Language Model Fine-tuning(LMF)**, and **Inconsistency Determinator(ID)**. We provide an overview of Vul_{diff}Finder and introduce each component afterward.

3.1. System workflow overview

The framework initially receives the extracted aspects and distinguishes it as an "atomic unit" or "composite structure" via the **AD**. It then categorizes the latter into "main" and "attachment", which exclusively comprise "atomic units". Subsequently, the **ID** is executed individually based on the syntactic hierarchical relationship of each "atomic unit" within "main" and "attachment", merging each level's results to derive the conclusion. The **Determinator** heavily relies on accuracy embedding text. To enhance embedding vulnerability knowledge, the framework leverages the aspect texts and original reports to fine-tune the SpanBERT model, which is realized by the **LMF**.

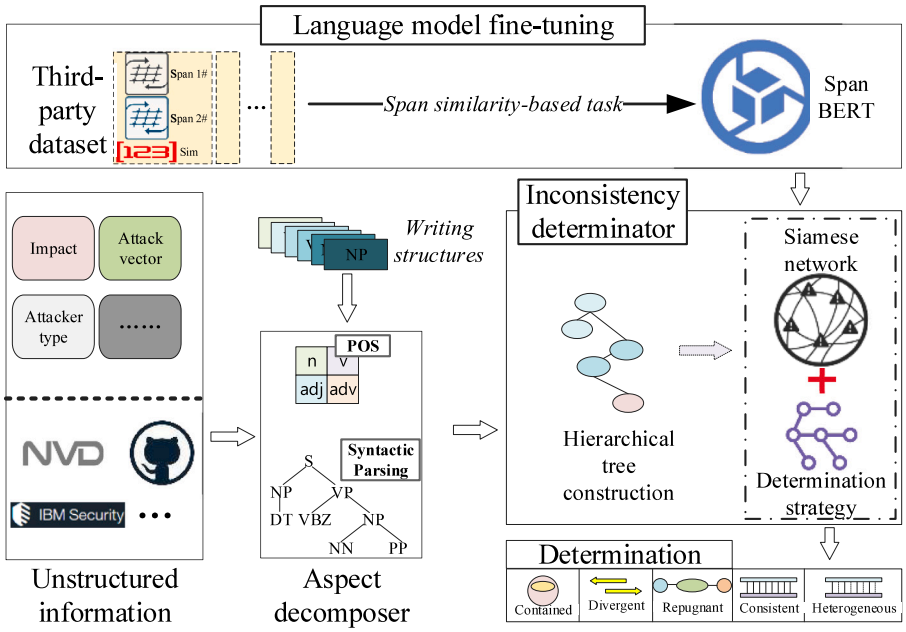


Fig. 2. The architecture of Vul_{diff}Finder.

3.2. Aspect decomposer

3.2.1. Writing structure categories

There is a large amount of free-form, unstructured information in vulnerability reports, represented by root cause, attack vector, impact and so on. By observing the writing and syntactic structure of these messages, it is found that most of them exist in the form of a span, and certain categories of messages have distinctive features in terms of syntactic components, e.g. root cause often starts with a verb, and the attacker type is mainly based on noun phrases. With this in mind, we have defined six categories of writing structure.

* **Word**: The text cannot be divided by space, as shown the *VN* in Gentoo in Fig. 1 .
* **Noun Phrase(NP)**: The text (1) consists of a noun with modifiers, or (2) connects multiple nouns using conjunctions (e.g., "and", "or") or punctuation indicating juxtaposition (e.g., "",), as shown the *VAT* in IBM.
* **Verb-Object(VO)**: The text starts with a verb and no prepositional adjunct follows the direct object, as shown the *VR* in NVD. Note that, juxtaposed verb-object structures are also considered as a "VO".
* **Word with Attachment(WwA)**: The aspect's subject is a single word (noun or cardinal number) and is : (1) accompanied by a connected constituent introduced by a preposition, or (2) followed by a word's POS tag included in ["IN","ADJ"] or special symbol like "≤", as shown the *VV* in NVD.
* **Noun Phrase with Attachment(NPwA)**: The aspect's core noun: (1) contains an adjective in front, (2) follows a modifier connected by a preposition or a "verb + object" structure, as shown the *VRC* in IBM.
* **Verb-Object with Attachment(VOwA)**: The text starts with a verb and the object is succeeded by an attached constituent with a preposition, as shown the *VAV* in Gentoo.

The initial two are characterized by naive and termed as "atomic unit", while the latter comprise intricate, denoted as "compound structure".

3.2.2. Decomposition principles

The **AD** recognizes the aspect's structural category and decomposes composite structure into atomic units. Given an aspect $aspect_t$, it utilizes NLP toolkits (i.e., Spacy and NLTK) for processing Constituency Parsing(**CP**) and Part-of-Speech tagging(POS). Each category's criteria include:

* If $split(aspect_t,\text{' '}) == 1$, it is a **Word**.
* If $aspect_t$'s subject consists of a word and accompanies a connected constituent introduced by a preposition or preceded by a modifier such as adjective, it is a **WwA**.
* If the *nltk.pos_tag* of $aspect_t$ starts with a verb(i.e., tagged as: VB, VBZ, VBP, VBD, VBN) or the *spacy.dep_tag* of "ROOT" refers to a verb, and no prepositional adjunct follows the direct object, it is a **VO**.
* If the *nltk.pos_tag* of $aspect_t$ starts with a verb and the direct object is succeeded by an attached constituent with a preposition or a "PART", it is a **VOwA**.
* If the $aspect_t$ proceed by *spacy.noun_chunks* output unique element or multiple juxtaposed ones (joined by ",", "and", etc.), it is a **NP**.
* If the unique result of *spacy.noun_chunks* is accompanied by a modifier, it is a **NPwA**.

The **AD** combines Spacy's Dependency Parsing (*spacy.token.dep_*) to further decompose composite structures. Regarding as **VOwA**, it labels words whose dependency tag is "root" and POS tag belongs to the verb, the constituents dependency with "verb" belongs to *[dobj,iobj,pobj]* labeled as "object", both of which are marked as "main". Constituents following the "object" and directly dependent on it are termed "attachment", and systematically broken down into "atomic units" based on criteria involving categorizing and decomposition. For **WwA** and **NPwA**, the word tagged as "ROOT" or the noun phrase containing "ROOT" are also labeled as "subject" through **DP** and **CP**. The remains are designated as "attachment" and subsequently decomposed into "atomic units".

3.3. Inconsistency determinator

The **ID** is responsible for determining the inconsistency among texts with various structures, such as Word vs NP, VO vs NPwA, NP vs NP, .etc. As previously discussed, the composite structure can be

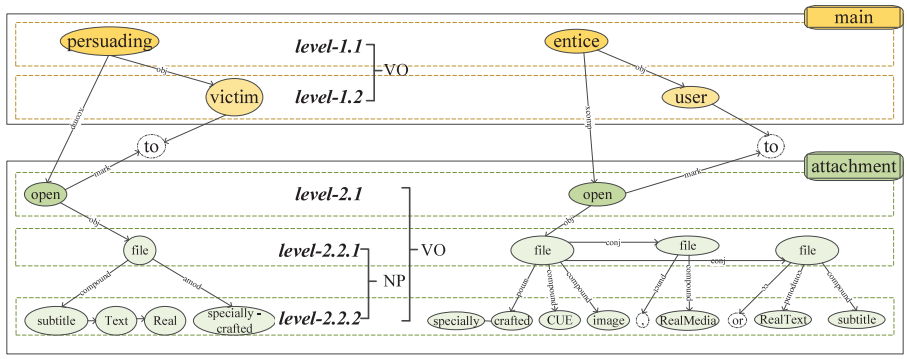


Fig. 3. Example of syntactic hierarchical tree construction based on the extracted "attack vector" of CVE-2008-5036 from IBM and Gentoo.

decomposed into "main" and "attachment" that contain atomic units, which exhibit syntactic relationships. Given this, we designed a hierarchical consistency determinator that integrates syntactic structures, comprising primarily two steps: *syntactic hierarchical tree construction* and *level-by-level inconsistency determination and integration*.

3.3.1. Syntactic hierarchical tree construction

The "attachment" in composite structures serves as a "complement" or "gerund" to the "main", which is linked by prepositions or "PART". Elements in the "main" are designated as **level-1**, while those in the "attachment" are **level-2**. We further process the "atomic units"-based composite in both parts. The hierarchical relationship between atomic units is constructed using preorder dependency (*spacy.token.head*). The "composite structure" iteratively employs preorder dependency until it encompasses all "atomic units", with those in the "main" being labeled as **level-1.x** and those in the "attachment" labeled as **level-2.x**.

As shown in Fig. 3, the "main" in IBM is *"persuading victim to open specially-crafted RealText subtitle file"*, which is a "verb+object" format and can be split into a verb of "persuading" and an object of "victim", labeled as **level-1.1** and **level-1.2**, respectively. The *"open .."*. is the "attachment" of *"victim"* as it serves as the object complement, which is a "verb+object" with an NP-style "object", labeled them as **level-2.1, level-2.2.1, level-2.2.2**(as shown the left tree). Simultaneously, we perform the same operation on Gentoo's "attack vector" to generate the right tree.

3.3.2. Level-by-level inconsistency determination and integration

This module takes in the compared aspects' all "atomic units", and along with syntactic hierarchical trees. Progressing gauges the consistency between pairs of atomic units layer by layer from the tree's apex. Siamese network Ni et al. (2021) is selected due to their exceptional performance in discerning input consistency(as shown in Fig. 4). Each layer of input atomic unit pairs to the Siamese network to generate a value of similarity. Subsequently, the similarities across layers are weighted and integrated to yield a final consistency result, thereby rectifying the integrated loss based on cross entropy loss Wang et al. (2020), as illustrated in Eq. (1).

$$L = -\frac{1}{N}\sum_{i}\sum_{c=1}^{M} y_{ic} log(\sum_{j=1}^{p} \lambda_j \times Sim_{icj}) \tag{1}$$

where N represents the sample size, M represents the number of judgment categories, p represents the layer number of the syntactic hierarchical tree, and y_{ic} is a symbolic function equaling 1 if the two judgments are not "Divergent" and "Repugnant", and 0 otherwise. Sim_{icj} represents the probability of the jth layer of the ith pair of samples being categorized as C. We employ cosine similarity as the measure of probability, where λ_j signifies the weight assigned to the jth layer. Bi-GRU was chosen to capture the global feature of each layer's information within the overall aspect, and the weights are shared between the two networks.

The two trees depicted in Fig. 3 serve as examples. Within the atomic unit of "main", two similarity values of sim-1.1 and sim-1.2 are established for **level_1.1** and **level_1.2**, respectively(i.e., <*"persuading" vs "entice"*> and <*"victim" vs "user"*>). These comparisons will so on down to subsequent levels like **level-2**, where similarities will be computed and integrated to adjust the constructive loss function. Additionally, several determination strategies warrant attention:

(1) The two **level-2.2** contain one and three **NPs**, respectively, and are determined in pairs.

(2) To determine NP pairs, we determine inconsistency in the order of nouns and modifiers.

(3) The inconsistency result of the preceding layer holds a "vote power" over the subsequent layer, i.e., if the two nouns in the preceding layer exhibit "Divergent" or opposing contexts, determination for the next layer ceases immediately.

(4) If one of the two determination terms within the same layer is empty, the determining for the subsequent layer is halted immediately.

These strategies enable it to determine the inconsistency between aspects with different writing structures simultaneously.

3.4. Language model fine-tuning

Accuracy embedding is crucial for determination. However, language models for accurately representing vulnerability domain knowledge are currently vacant. Thus, we use transfer learning to fine-tuning existing language models to handle it. As aspects are mainly expressed in multi-word (i.e.: phrases), we choose SpanBERT[3] as the base. For fine-tuning task, we refer to the Sentence Similarity Task (STS-B) in GLUE and set up a span similarity-based task, it includes *span vectorization*, *similarity calculation* and *loss function*.

Span vectorization. SpanBERT leverages the mean value of the tokens' vector within the span or boundary values to represent the span. It ensures equal contribution from each token in the span, a departure from conventional practice. Hence, we assign varying weight combinations to words with diverse POS within the span, as illustrated in Eq. (2).

$$E_{span} = \frac{\sum_{i=1}^{len}(w^{pos} \times E_i)}{len} \quad w^{pos} \in [w^{verb}, w^{noun}, w^{adv}, w^{adj}, w^{o}] \tag{2}$$

where len denotes span's length. Assigning weight for each token within the span according to the POS, resulting in the embedded representation of the span is derived through a weighted average.

Similarity Calculation. The similarity between vectors is commonly expressed with Euclidean Distance Chandrasekaran and Mago

[3] SpanBERT: https://github.com/facebookresearch/SpanBERT.

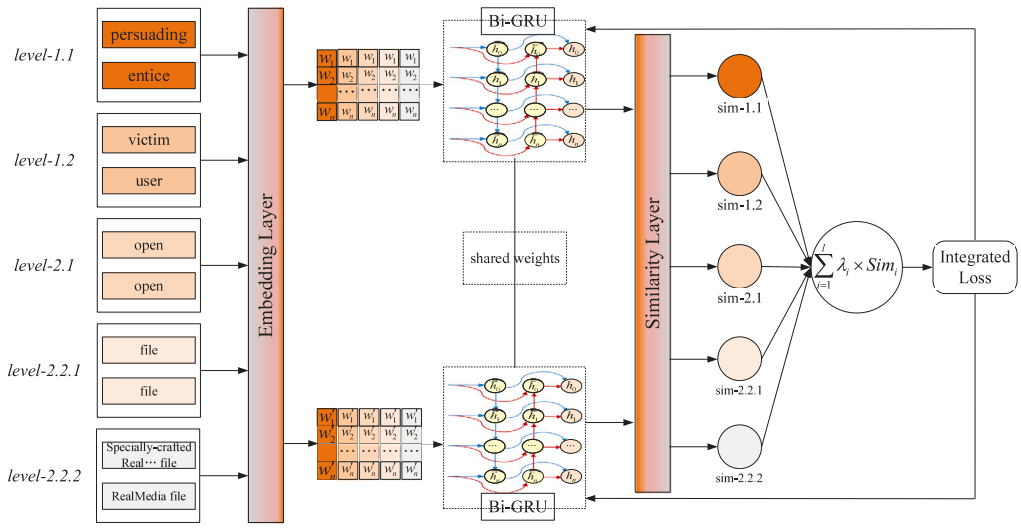


Fig. 4. Architecture of the Siamese Network with Integrated Loss.

(2021), cosine similarity, and so on. We consistently employ Euclidean Distance for measuring vector similarity, as illustrated in Eq. (3).

$$sim_{v_1,v_2} = \sqrt{\sum_{k=1}^{m} (y_k^{(t)} - y'^{(t)}_k)^2} \tag{3}$$

where m denotes the dimension. Given the presence of domain-specific terms, such as "DoS" and "denial of service", we created alias dictionaries (referred as *product_dictionary* and *vul_type_dictionary*) for *VN* and *VT*. The similarity among aliases is 1.

Loss Function. Conventional loss functions for text similarity tasks encompass MSE, MAE, and others. The advantages and disadvantages of both methods are obvious. Thus, we opt for the Huber Loss Wang et al. (2020) that amalgamates the strengths of MSE and MAE, to assess the disparity between the predicted and actual similarity, as depicted in Eq. (4).

$$L_\delta(y, \overline{y}) = \begin{cases} \frac{1}{2} \times (y - \overline{y})^2, & for|y - \overline{y}| < \delta \\ \delta \times |y - \overline{y}| - \frac{1}{2}\delta^2, & otherwise. \end{cases} \tag{4}$$

where $\overline{y}$ is the predicted similarity and δ is the hyperparameter. The Fine-tuned SpanBERT enables it accurate represent spans within vulnerability reports, serving as quality assurance for subsequent fine-grained consistency measures.

4. Argumentation

This section discusses the datasets, experimental schemes, and corresponding results.

4.1. Datasets

This framework requires datasets including: the *domain-specific span-similarity dataset* for fine-tuning the SpanBERT model and the *aspect-inconsistency dataset* for validation.

4.1.1. Domain-specific span-similarity dataset

The aspect_discrepency dataset[4] identifies discrepancies in identical aspect across sources. It encompasses 3385 entries with pairs of *VRC*, *VAV*, *VT*, and *VR*. Each entry comprises text from two sources and their inconsistency label ("0" or "1"). Text pairs initially labeled as "1" retain their labels in each short text pair. As for that with "0", we form multiple shorter pairs by decomposing the long texts into atomic units, select the corresponding constituent pairs and evaluate their consistent labels jointly by three cybersecurity researchers.

4.1.2. Aspect-inconsistency dataset

The aspect_discrepancy dataset comprises vulnerability aspects that exhibit one-sidedness, and the granularity of the labeling results is coarse, rendering them inapplicable to our task. We focus on NVD and extend the data sources by analyzing their reference links. On the one hand, domain-specific (IOT-related) vulnerability reports are selected, and on the other hand, generic vulnerability reports (e.g. IBM, Exploit-DB) are also selected to ensure the universality of the dataset. In the end, we chose four sites related to the IOT domain, namely GitHub,[5] Packet Storm.[6] and VULHUB[7] Meanwhile, four open-source sites for generic vulnerabilities are chosen, namely Exploit-DB, IBM X-force, Gentoo, and KBCert. We manually annotated eight information categories (refer to Section 2.1) within these reports, then paired identical aspects in two reports and recorded a consistent result. Consequently, we compiled a dataset comprising 1612 pairs of aspects alongside their corresponding consistency results.

4.2. Experimental schemes

This section describes the experimental scheme and evaluation metrics used to validate the effectiveness of the proposed method. First, since Vul_{diff}Finder consists of several pipelined execution processing modules, an ablation experiment is designed to validate the impact of each module on the final determination result, detailed in Section 4.2.1. Then, to validate the advantage of the method proposed in this paper, a comparison with the state-of-the-art information inconsistency determination scheme is selected, and the reasons for the chosen scheme and its introduction are described in Section 4.2.2. Finally, the evaluation metrics used to verify the effectiveness of each experimental scheme are also presented, as shown in Section 4.2.3.

[4] aspect_discrepency dataset: https://github.com/anonymous-dev905/aspect_discrepency/tree/master/Labels/labels_discrepancy_classifier

[5] Github: https://github.com/.

[6] Packet Storm: https://packetstormsecurity.com/.

[7] VULHUB: https://www.scap.org.cn/view/iot.

4.2.1. Ablation experiments

This section focuses on verifying the effectiveness of each module in improving the performance of inconsistency determination.

* We validate the performance with and without AD, denoted as **Vul_{diff}Finder-with-AD** and **Vul_{diff}Finder-without-AD**. Once AD is omitted, inconsistency is determined solely based on the whole text of the aspect, without employing a hierarchical strategy.
* Concerning LM fine-tuning, we validate the performance directly using pre-trained and fine-tuned language model, denoted as **Vul_{diff}Finder-with-LM-fine-tuning** and **Vul_{diff}Finder-without-LM-fine-tuning**.
* As for the Inconsistency Determinator, we argue its determination performance with and without the use of a hierarchical determination strategy, which are expressed as **Vul_{diff}Finder-with-Hie-strategy** and **Vul_{diff}Finder-without-Hie-strategy**.

4.2.2. Baseline models

Detecting inconsistencies in unstructured vulnerability aspects poses significant challenges, with limited dedicated studies. Hence, we select the most representative schemes for comparison.

* **ADD**: Sun (Sun et al., 2023c) aims at the task of vulnerability aspect discrepancy detection, with techniques like Glove, Siamese network and provide foundational dataset.
* **LLM**: The large language model adeptly determines inconsistency among texts, so we choose the GPT-Turbo 3.5 model.
* **SBERT**: Some studies treat the determination as a classification task and opt for the BERT model or its variants (e.g., Sentence BERT) to achieve end-to-end determination. Thus, SBERT will serve as our baseline.

4.2.3. Evaluation metrics

Two metrics are necessary for evaluating our work. Firstly, to ascertain the disparity between Vul_{diff}Finder's extraction performance and the ground truth, we employ precision, recall, and F1-score as evaluation metrics for both the redundancy and baseline model experiments. Secondly, to globally illustrate the quantitative results of the determination results of the framework in terms of information inconsistency, Eq. (5) serves as the metric.

$$aspect_{ij} = \frac{\sum_{k=1}^{N} (tag_k == j)}{N} \tag{5}$$

Where $aspect_{ij}$ is the percentage of the ith aspect belonging to category j, N denotes the amount of aspect pairs, and tag_k is the determination label.

4.3. Experiment results

4.3.1. Effectiveness of aspect decomposer

The AD facilitates the consolidation and streamlining of textual and composite structures into atomic units. Significant disparities exist in the writing structures among various aspects, and the utility provided by AD varies greatly. Table 2 illustrates the statistical distribution of writing structures across different aspects.

The table indicates that "word" exhibits lower distribution across all aspects as its limited informational content. "NP" predominantly occurs in aspects like *VN*, *VT*, *VAT*, and *VP*, comprising proper nouns and domain-specific phrases. Conversely, "VO" and "VOwA" are characterized by aspects such as *VR*, delineating the behavior type, subject, and object. "NPwA" primarily appears in aspects like *VRC*, *VP*, and *VAT*, necessitating further description (e.g., "unauthenticated", "with root privilege") of cybersecurity subjects ("attackers", file name). "WwA" mainly observes in aspects such as *VN*, *VAT*, which necessitate other terms to qualify subject (e.g., "below", "<").

We conducted a comparison of the overall and detailed determining performance with and without the AD. Noted that the F1-score serve as metric for detailed comparison,as shown in Fig. 5.

According to the results, the performance with the AD is significantly better, exceeding 3.48% improvement compared to without. This results from AD reducing complex structures into atomic units and combining with a hierarchical decision strategy to achieve a "softer" consistency decision. Moreover, it can circumvent inaccuracies in embedding language models when handling lengthy text. In detailed comparison, AD shows a negligible improvement (around 1%) on word-based aspects such as *VN* and *VV*, also proper nouns and specific phrases-based aspects like *VT* and *VAT*. Yet, when handling aspects like *VRC*, *VAV*, and *VR* that comprise structures like "VO" and "VOwA", AD simplifies these into atomic units, resulting in a substantial improvement (averaging about 6%). Additionally, AD can also strip out of the vulnerability's location from multi-layer nested *VP*, pinpointing high-value information accurately within complex text and enhancing performance (by 3%).

4.3.2. Effectiveness of LM fine-tuning

The accurate embedding ability of the language model is crucial for assessing the similarity among texts. To address the current deficiency in vulnerability domain-specific language models, we fine-tune SpanBERT and showcase its capacity to enhance embedding accuracy at both macro and micro scales. During the fine-tuning, we restructure the span's embedding vector according to POS weighting. We focus on nouns, verbs, adjectives, and adverbs as they carry significant information in the text. Simultaneously, we also focused on adposition like "below", "through", as it is particularly important in indicating the scope of cybersecurity subjects (e.g., "version", "attacker type") . Their weights are obtained based on the TFIDF method, and the details are shown in Table 3.

It reveals that verbs and nouns (including proper nouns) carry the greatest significance due to pivotal role at semantic level. Following is Adposition, which indicating scope and pertains specifically to the vulnerability domain. Subsequently, Adverbs denoting degree and hold slightly less weight, while Adjectives carry lesser weight. The others make minimal contributions and possess the lowest weight.

We select representative text pairs about vulnerability in Table 4 to assess the fine-tuned model's improvement at a microscopic level.

We demonstrate the efficacy of fine-tuning by the alterations in semantic similarity. Additionally, we compare the non-fine-tuned model and that employing weighted averaging. As shown in Fig. 6.

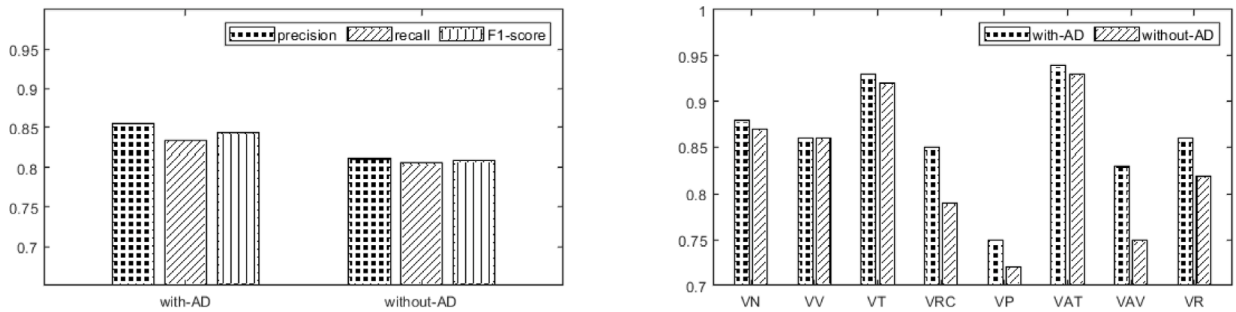
Overall, the fine-tuned model demonstrates superiority compared to the unfine-tuned model, and the POS-based weighted method outperforms the direct weighting approach. The enhancement of fine-tuning is limited for the Word, and weighting methods specific for span also exhibit no discernible effect. As for the VO, variations within spans reveal a significant distinction, particularly due to verb ("gain" vs "execute") and noun ("privileges" vs "code") similarities, which are inconsistent. The POS-based approach assigns greater weight to both POS types, which accentuates the disparity. In the NP, the diverse span combinations significantly influence outcomes, evident in noun agreement ("attacker" in both) and clear distinctions in adjectives ("remote" vs "local"), magnifying textual differences. The limited impact of fine-tuning may stem from the generic nature of constituent words. In WwA, VOwA, and NPwA, longer texts necessitate prioritizing key terms with higher weights, a strength of the POS-based approach, and results in favorable outcomes. Simultaneously, fine-tuning semantic adjustments for domain-specific terms (e.g., "implementation", "handling", and "unauthenticated") accentuates textual disparities.

We also select fine-tuned and non-fine-tuned language models to compare their performance, with specific results depicted in Fig. 7.

The results demonstrate a slight improvement in performance when utilizing the fine-tuned language model compared to the other, with

Table 2
Statistics on the proportion of each aspect's writing structure.

Category	VN	VV	VT	VRC	VP	VAT	VAV	VR
Word	0.153	0.163	0.0142	0.0149	0.0948	0.0196	0.0172	0
NP	**0.7258**	0.2212	**0.6112**	0.0944	**0.3956**	**0.3834**	0.2164	0.132
VO	0	0.0066	0.0393	0.0456	0.0164	0	0.2013	**0.4433**
WwA	0.0167	**0.5957**	0.0867	0.1775	0.2116	**0.3236**	0.1143	0.0206
NPwA	0.1043	0.0085	0.2285	**0.4621**	0.22935	0.2734	0.1365	0.0486
VOwA	0	0.0048	0.0201	0.2055	0.0523	0	**0.3143**	0.3554



(a) Comparison of overall performance (b) Comparison of detailed performance in each aspect

Fig. 5. Comparison of Inconsistency Determination Performance With and Without the Aspect Decomposer.

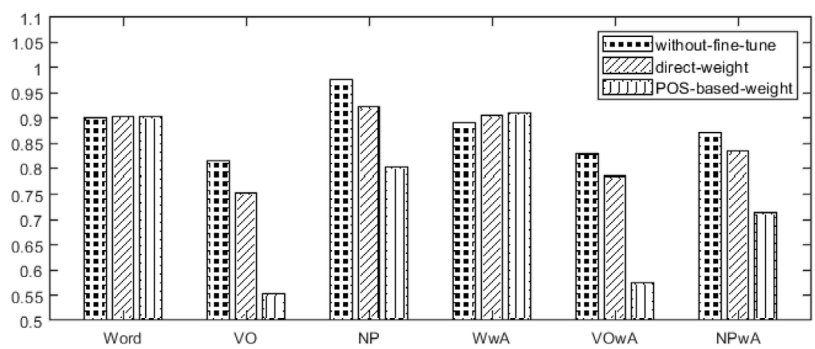


Fig. 6. Comparison of Semantic Similarity With and Without the LM Fine-tuning.

Table 3
Each POS and its weight.

POS	Verb	Noun	Adposition	Adjective	Adverb	Other
weight	0.47	0.19	0.15	0.11	0.04	0.02

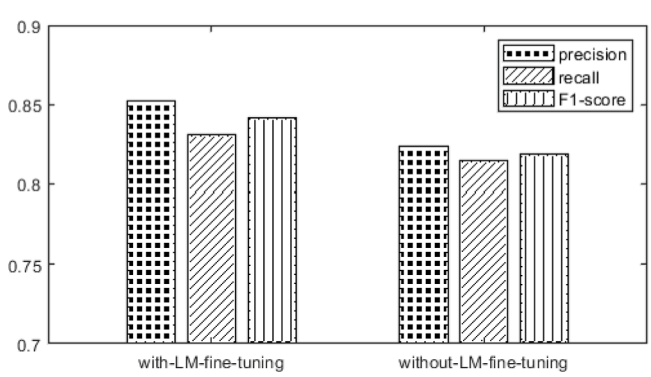


Fig. 7. Comparison of Inconsistency Determination Performance With and Without the LM Fine-tuning.

an increasing from 81.89% to 84.15% for F1-score. In-depth analyses uncovered the following factors:

(1) The inspect_discrepancy dataset is scientific annotation by five researchers to ensure its reliability. We refined the dataset to align with our goal's specific requirements, enhancing its usability.

(2) Incorporating Span embedding with POS weighting emphasizes the significance of words with POS like nouns and verbs, which are pivotal in vulnerability aspects. It enhances the language model's capability to articulate crucial words.

(3) The fine-tuned dataset's minimal inclusion of proper nouns and out-of-vocabulary (OOV) words, consequently gets rid of impeding the module's capacity to improve its performance.

4.3.3. *Effectiveness of inconsistency determinator*

The Inconsistency Determinator consists of two parts: syntactic hierarchy tree construction and determination strategy. The syntactic hierarchical trees of each aspect show different layers for "main" and "attachment" respectively, as shown in Fig. 8.

Significant discrepancies exist in the hierarchical structures among different aspects. For instance, the *VV* and *VAV* primarily governed by the "Word" and "WwA", respectively. The "attachment" of the former primarily consists of adpositions, restricting the scope of the "main" with fewer hierarchical levels. Conversely, the latter offers a detailed description of the "main" aspect at a higher level. *VRC* and *VR* utilize the "VO" as "main", encompassing a complete verb and object. Their "attachment" serves as a complementary description of the object's

Table 4
Text pair samples with various writing structures.

Type	Text1	Text2
Word	"v6.0-1"	"6.01"
VO	"gain privileges"	"execute arbitrary code"
NP	"remote attacker"	"local attacker"
WwA	"handling of HNAP login requests"	"implementation of the HNAP"
VOwA	"execute arbitrary commands with root privileges"	"gain arbitrary code execution as root"
NPwA	"remote attacker with network access via HTTPS"	"unauthenticated attackers with network access"

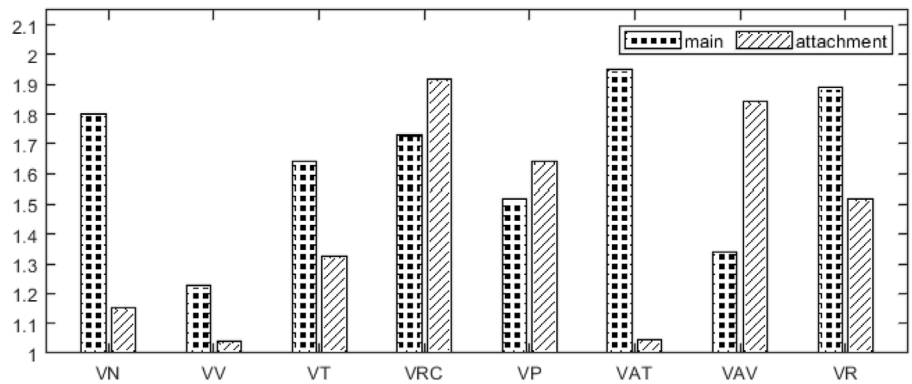


Fig. 8. Hierarchical tree height statistics for each aspect on "main" and "attachment".

Table 5
Performance with different hyperparameters.

Weight of the "main"	λ_1	λ_2	λ_3	λ_4	λ_5	F1-score
0.8	0.64	0.16	0.16	0.032	0.008	0.8353
0.85	0.68	0.17	0.12	0.024	0.006	**0.8415**
0.9	0.72	0.18	0.08	0.016	0.004	0.8389
0.95	0.76	0.19	0.04	0.008	0.002	0.8362

behavior, exhibiting a higher level. The *VN*, *VAT*, and *VT* feature a distinct noun phrase as a primary component and exhibit hierarchical layers. The "attachment" is used to constrain the "main" with phrases like "with network access" and "via HTTPS", which exhibit lower levels of hierarchy. Lastly, *VP* primarily comprising network entities with files and applications, features an "NP" as "main" and indicates a higher hierarchical level. Its attachment primarily offers a supplementary description of the "NP", typically through adjectives, resulting in a lower hierarchical level.

Additionally, fusing the determination results of each level involves the setting of the hyperparameter λ_j. The preceding statistic indicates a main distribution of 2 layers for "main" and 3 layers for "attachment". Consequently, we establish 5 hyperparameters of λ. Specifically, λ_1 and λ_2 denote the primary and secondary levels of "main", while λ_3 through λ_5 denote various levels of "attachment". Adhering to the principle that "higher levels possess precedence over lower ones", we posit that higher levels correlate with larger parameters. Therefore, based on the "Pareto principle", we allocate an initial weight of 0.8 to the "main", distributing 80% to its first layer and 20% to the second. This procedure is replicated for "attachment". Subsequently, we conduct a grid search incrementing by 0.05 of the weight of the "main" to ascertain the optimal hyperparameters, as shown in Table 5.

The results indicate that as the weight of "main" increases, the F1-score initially rises, then declines. Optimal performance is attained at [0.68, 0.17, 0.12, 0.024, 0.006] for each λ. We further verify the effectiveness of the determination strategy, i.e., the comparison of the determination performance with and without the hierarchical determination strategy, as shown in Fig. 9.

The results demonstrate a significant improvement when employing the hierarchical determination strategy. When without the strategy, decisions rely solely on the semantic information of the span and result in identical contributions for each word in the span to the decision, which is unreasonable. This strategy initially identifies the "main" that comprises nouns and verbs, and assigns them greater importance. Subsequently, it identifies "attachment" to ensure distinct components exhibit inequally influence. This aligns with the human inclination to assess "keys" before "modifiers".

4.3.4. Compared with other works

We compare Vul_{diff}Finder with state-of-the-art information inconsistency determination schemes. Fig. 10 demonstrate that Vul_{diff}Finder outperforms other schemes significantly (with an F1 value 4.31% higher than the top-performing ADD scheme). As it utilize the AD for finer-grained comparison units compared to ADD, enhancing domain knowledge representation through fine-tuning language model rather than employ models like Glove directly. Besides, ADD employs a "hard" determination method like WMD or an unoptimized Siamese CNN network, complicating accurate determination for texts with complex and variable structures. SBERT performs the poorest among models (with an F1-score of 71.72%) as it lacks fine-tuning and relies solely on the pre-trained model for determination. Additionally, SBERT typically processes complete sentences, whereas this work takes phrases, words, or clauses as input, which also affects the model's performance due to input disparity. LLM demonstrates strong performance across NLP tasks, while exhibits poor performance for this task (with an F1-score of 74.08%). Analysis revealed that LLM's responses were overly ambiguous when provided with explicit category boundaries, often categorizing semantically different entities as "Heterogeneous" or "Contained".

4.4. Result analysis

The distribution of inconsistency results across aspects was quantified and presented in Table 6, which reveal several noteworthy phenomena.

* "Consistent" exhibits the highest occurrence rates in *VN*, *VRC*, *VP*, *VAT*, *VAV*, and the second in *VV*, *VT*, *VR*. It indicates the circulation of vulnerability information among sites, as is the only way to ensure intelligence consistency in writing and semantic.

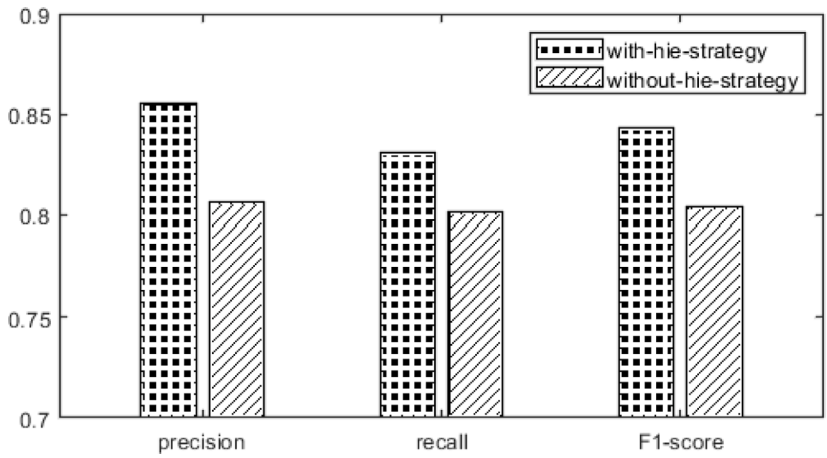


Fig. 9. Performance Comparison of With and Without Hierarchical Determination Strategy.

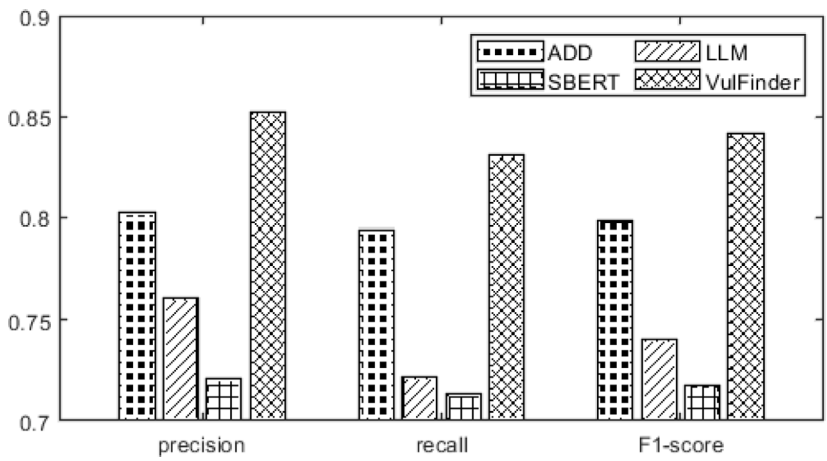


Fig. 10. Comparison of Inconsistency Determination Performance Among the Vul_{diff}Finder and Other works.

Table 6
Inconsistency results statistics for each aspect.

Category	VN	VV	VT	VRC	VP	VAT	VAV	VR
Consistent	41.66%	28.32%	35.24%	44.16%	48.54%	56.46%	30.53%	19.87%
Heterogeneous	23.33%	19.53%	7.35%	19.65%	5.24%	1.65%	21.54%	14.05%
Contained	27.24%	30.37%	23.65%	10.11%	28.12%	22.43%	11.55%	38.30%
Divergent	4.47%	21.83%	32.93%	27.14%	9.32%	14.65%	34.58%	24.50%
Repugnant	3.32%	0%	0.90%	0%	8.83%	4.88%	1.82%	4.33%

* "Contained" distributed in aspects like *VN* and *VT* due to the comprehensive disclosure of information by sites (e.g., vendors, required permissions). It occurs in *VV* and *VR* attributes to sites disclosing varying validation conditions or observations. Additionally, it appears in *VP* as some sites describe more comprehensive file paths, which are key to locating vulnerabilities.
* "Heterogeneous" mainly occurs in *VV*, *VRC*, *VAV*. The first involves approximations like "-", "to" and "through" to indicate the scope variance, while the latter two reflect the differ in expressing behavior.
* "Divergent" is distributed in *VT*, *VRC*, *VAV*, *VR*, representing various aspects expressing observable phenomena in the vulnerability validation process. The disparities among these aspects mirror the differences in how vulnerabilities are validated and the focus of observations across different sites.
* "Repugnant" exhibits the lowest frequency, with the highest occurrence rates being merely 8.83% for *VP* and 4.88% for *VAT*. As the information disclosed by each site is closely verified and referenced to other open sources, making it difficult to draw contradictory conclusions. This inconsistency is pivotal in affecting information quality, given the challenge for intelligence consumers face in making binary decisions.

Overall, without the "Consistent", there reveals differences in the completeness of information conveyed by the "attachment" of *VN*, *VV*, *VP*, and aspects with "NPwA" and "WwA" as the primary structure, rendering it mainly "Contained". The *VR* characterized by "NP" and "NPwA" exhibits similar incomplete information in its main and attachment, also be classified as "Contained". Regarding *VT* and *VAT*, primarily relying on proper nouns, diverse evaluations conducted by various platforms yielded discrepant outcomes concerning the vulnerability's key components, predominantly categorized as "Divergent". Concerning *VRC* and *VAV*, delineating the intricate behavior of the vulnerability, the format and emphasis of behavior description varied across platforms. Furthermore, *VV* and *VAV* also exhibit "Heterogeneous".

5. Discussions

In this section, we discuss the implications and future work of our search:

5.1. Implications

The work in this paper has been effective in determining the semantic consistency of unstructured vulnerability information, and its implications can be summarized in two main areas.

(1)Domain-Specific Language Model: the pretrained language models are less expressive for specialized texts. We compile a dataset that consists of identical vulnerability aspect pairs from multiple sources along with their similarities. Simultaneously, we fine-tune the SpanBERT model by creating span-level text embedding expressions that incorporate POS to accuracy embedding aspects.

(2) Inconsistency determination strategy for free-writing text pair: vulnerability aspects are often structurally complex, rendering existing strategies for handling inconsistency in structured data. We address inconsistency in complex texts by breaking them down into multiple "atomic units" and developing determination strategies for various types of text structure pairs.

Our research has made promising progress in the inconsistent measurement of unstructured vulnerability information, but further work is needed to address the legacy limitations in improving the determination accuracy and intelligent handling of contradictory information.

5.2. Future work

We propose three areas for future work. First, the pipelined processing in the current system architecture may introduce errors. Therefore, one focus of our future work will be on designing an end-to-end architecture aimed at minimizing processing flow and enhancing the system's automation and decision-making capabilities. Second, we are currently challenged by the difficulty of determining whether multi-source information is contradictory or mutually exclusive. We have been unable to achieve "truth discovery" in the presence of conflicting information. Thus, our second area of future work will concentrate on automating the identification of mutually exclusive information. Finally, our research aims to facilitate the practical application of consistency determination results. This includes assessing the quality of information sources based on consistency outcomes, reducing redundancy and misinformation in multi-source data, and integrating and optimizing vulnerability intelligence.

6. Related work

6.1. Applications of vulnerability report

Vulnerability aspects contained in open-source reports is valuable in aiding many downstream cybersecurity tasks. Mu (Mu et al., 2018) demonstrated that the absence of information in vulnerability reports diminishes the success rate of vulnerability reappearance. Dong (Dong et al., 2019) extracted information of "affected product" and "version" from multi-source vulnerability reports and measured their inconsistency. Zhang (Zhang et al., 2015) employed NVD data to conduct security risk assessment. Binyamini (Binyamini et al., 2021) extracted vulnerability key aspects from vulnerability reports to construct an attack graph for network security assessment. Ruchi (Sharma et al., 2021) and Han (Han et al., 2017) utilized vulnerability descriptions to prioritize vulnerabilities, and Guo (Guo et al., 2021, 2020) extracted critical information from vulnerability reports and predict missing information. Existing research has largely focused on extracting unstructured information from vulnerability reports or using structured information for downstream cybersecurity tasks, such as cyber risk assessment, attack scenario reconstruction, and threat detection. Unfortunately, there has been little emphasis on evaluating the quality of these information. The quality of the information significantly impacts the effectiveness of its application. Therefore, our goal is to measure the inconsistencies among multi-source vulnerability information to enhance the overall quality of vulnerability information.

6.2. Vulnerability information inconsistency analysis

In practical systems of data integration, multiple sources of data being integrated often offer inconsistent or divergent descriptions of identical information, which is also a common occurrence in vulnerability reports. Dong (Dong et al., 2019) identifies inconsistencies in products and versions in multi-source vulnerability reports. Jiang (Jiang et al., 2021) investigate the inconsistency of impact among databases on real-world IT/OT infrastructures, including dimensions of disclosure time, CPE and CVSS, by five security experts. Nappa (Nappa et al., 2015) compares the versions disclosed by Mozilla and NVD, and identifies issues in the disclosed versions from NVD primarily related to missing versions and irrelevance. Anwar (Anwar et al., 2021) investigates inconsistencies between software vendors, severity, and vulnerability type utilizing NVD as base. Farhang (Farhang et al., 2020) investigates the inconsistency of vulnerability severity disclosure between Android security bulletins and mainstream vendors' security bulletins. Sun (Sun et al., 2022) built matching models based on aspects like type, impact, and attack vector from NVD and Bugzilla vulnerability reports to support the recovery of traceability relationships between vulnerabilities. Sun (Sun et al., 2023b) also integrates key aspect information from heterogeneous databases based on word2vec and kmeans to build an aspect-level knowledge graph. Kuehn (Kuehn et al., 2021) measures the difference in accuracy, completeness, and uniqueness of structured information like scope and other 13 attributes in NVD. The current research primarily focuses on vulnerability information presented in a simple structure and regular writing format, allowing for the identification of character-level inconsistencies, such as misspellings and numerical discrepancies. In contrast to existing studies, this research emphasizes unstructured vulnerability information and assesses inconsistencies at the semantic level. This approach significantly expands the range of vulnerability information that can be effectively processed.

7. Conclusion

The prevalence of information heterogeneous and conflict across vulnerability reports significantly affects vulnerability intelligence's usability and quality. This paper proposes a method for determining inconsistencies of unstructured vulnerability aspects. It utilizes multiple NLP tools to simplify complex texts into structurally simple units, and designs span similarity-based tasks to fine-tune SpanBERT for accuracy embedding vulnerability knowledge. Additionally, a hierarchical determination strategy is designed to address aspects with diverse writing structures. The scheme is thoroughly validated using the constructed dataset, and the results demonstrate the superiority of the proposed method. The experimental results also indicate that information inconsistency is prevalent and varies across aspects. Nevertheless, there is still room for improvement. For instance, pipelined processing may introduce errors and be a crucial issue for future research.

CRediT authorship contribution statement

Qindong Li: Writing – original draft, Visualization, Validation, Software, Methodology. **Wenyi Tang:** Writing – review & editing, Formal analysis, Conceptualization. **Xingshu Chen:** Supervision, Project administration. **Hao Ren:** Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research work was partly supported by the Young Scientists Fund of the National Natural Science Foundation of China under Grant (No. 62302322), the Fundamental Research Funds for the Central Universities, China (No. SCU2023D008), the Fundamental Research Funds for the Central Universities, China (No. 2022SCU12116), Sichuan University Postdoctoral Interdisciplinary Innovation Startup Fund, China Project (No. 10822041A2076), the Intelligent Policing and National Security Risk Management Laboratory 2023, China Opening Project Fund under Grant (No. ZHKFYB2304), the Science and Engineering Connotation Development Project of Sichuan University, China (No. 2020SCUNG129) and the Key Laboratory of Data Protection and Intelligent Management (Sichuan University), Ministry of Education, China.

Data availability

Data will be made available on request.

References

Anwar, A., Abusnaina, A., Chen, S., Li, F., Mohaisen, D., 2021. Cleaning the NVD: Comprehensive quality assessment, improvements, and analyses. IEEE Trans. Dependable Secur. Comput. 19 (6), 4255–4269.

Binyamini, H., Bitton, R., Inokuchi, M., Yagyu, T., Elovici, Y., Shabtai, A., 2021. A framework for modeling cyber attack techniques from security vulnerability descriptions. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. pp. 2574–2583.

Chandrasekaran, D., Mago, V., 2021. Evolution of semantic similarity—a survey. ACM Comput. Surv. 54 (2), 1–37.

Chaparro, O., Lu, J., Zampetti, F., Moreno, L., Di Penta, M., Marcus, A., Bavota, G., Ng, V., 2017a. Detecting missing information in bug descriptions. In: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. pp. 396–407.

Chaparro, O., Lu, J., Zampetti, F., Moreno, L., Di Penta, M., Marcus, A., Bavota, G., Ng, V., 2017b. Detecting missing information in bug descriptions. In: Bodden, E., Schäfer, W., van Deursen, A., Zisman, A. (Eds.), Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. ESEC/FSE 2017, Paderborn, Germany, September 4-8, 2017, ACM, pp. 396–407.

Cheng, Y., Yang, S., Lang, Z., Shi, Z., Sun, L., 2023. VERI: A large-scale open-source components vulnerability detection in IoT firmware. Comput. Secur. 126, 103068.

Dong, Y., Guo, W., Chen, Y., Xing, X., Zhang, Y., Wang, G., 2019. Towards the detection of inconsistencies in public security vulnerability reports. In: Heninger, N., Traynor, P. (Eds.), 28th USENIX Security Symposium. USENIX Security 2019, Santa Clara, CA, USA, August 14-16, 2019, USENIX Association, pp. 869–885.

Fan, Y., Xia, X., Lo, D., Hassan, A.E., 2020. Chaff from the wheat: Characterizing and determining valid bug reports. IEEE Trans. Softw. Eng. 46 (5), 495–525.

Farhang, S., Kirdan, M.B., Laszka, A., Grossklags, J., 2020. An empirical study of Android security bulletins in different vendors. In: Proceedings of the Web Conference 2020. pp. 3063–3069.

Guo, H., Xing, Z., Chen, S., Li, X., Bai, Y., Zhang, H., 2021. Key aspects augmentation of vulnerability description based on multiple security databases. In: 2021 IEEE 45th Annual Computers, Software, and Applications Conference. COMPSAC, IEEE, pp. 1020–1025.

Guo, H., Xing, Z., Li, X., 2020. Predicting missing information of key aspects in vulnerability reports. arXiv preprint arXiv:2008.02456.

Han, Z., Li, X., Xing, Z., Liu, H., Feng, Z., 2017. Learning to predict severity of software vulnerability using only vulnerability description. In: 2017 IEEE International Conference on Software Maintenance and Evolution. ICSME, IEEE, pp. 125–136.

Jiang, Y., Jeusfeld, M.A., Ding, J., 2021. Evaluating the data inconsistency of open-source vulnerability repositories. In: Reinhardt, D., Müller, T. (Eds.), ARES 2021: The 16th International Conference on Availability, Reliability and Security. Vienna, Austria, August 17-20, 2021, ACM, pp. 86:1–86:10.

Kuehn, P., Bayer, M., Wendelborn, M., Reuter, C., 2021. OVANA: An approach to analyze and improve the information quality of vulnerability databases. In: Reinhardt, D., Müller, T. (Eds.), ARES 2021: The 16th International Conference on Availability, Reliability and Security. Vienna, Austria, August 17-20, 2021, ACM, pp. 22:1–22:11.

Li, Q., Tang, W., Chen, X., Feng, S., Wang, L., 2024. Comprehensive vulnerability aspect extraction. Appl. Intell. 54 (3), 2881–2899.

Mu, D., Cuevas, A., Yang, L., Hu, H., Xing, X., Mao, B., Wang, G., 2018. Understanding the reproducibility of crowd-reported security vulnerabilities. In: Enck, W., Felt, A.P. (Eds.), 27th USENIX Security Symposium. USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018, USENIX Association, pp. 919–936.

Nappa, A., Johnson, R., Bilge, L., Caballero, J., Dumitras, T., 2015. The attack of the clones: A study of the impact of shared code on vulnerability patching. In: 2015 IEEE Symposium on Security and Privacy. IEEE, pp. 692–708.

Nguyen, V., Massacci, F., 2013. The (un) reliability of NVD vulnerable versions data: An empirical experiment on google chrome vulnerabilities. In: 8th ACM SIGSAC Symposium on Information, Computer and Communications Security. ASIA CCS 2013, pp. 493–498.

Ni, P., Li, Y., Li, G., Chang, V., 2021. A hybrid siamese neural network for natural language inference in cyber-physical systems. ACM Trans. Internet Technol. (TOIT) 21 (2), 1–25.

Sharma, R., Sibal, R., Sabharwal, S., 2021. Software vulnerability prioritization using vulnerability description. Int. J. Syst. Assur. Eng. Manag. 12, 58–64.

Sun, H., Ou, G., Zheng, Z., Liao, L., Wang, H., Zhang, Y., 2023a. Inconsistent measurement and incorrect detection of software names in security vulnerability reports. Comput. Secur. 135, 103477.

Sun, J., Xing, Z., Lu, Q., Xu, X., Zhu, L., 2023b. A multi-faceted vulnerability searching website powered by aspect-level vulnerability knowledge graph. In: 45th IEEE/ACM International Conference on Software Engineering: ICSE 2023 Companion Proceedings. Melbourne, Australia, May 14-20, 2023, IEEE, pp. 60–63.

Sun, J., Xing, Z., Xia, X., Lu, Q., Xu, X., Zhu, L., 2023c. Aspect-level information discrepancies across heterogeneous vulnerability reports: Severity, types and detection methods. ACM Trans. Softw. Eng. Methodol. 33 (2), 1–38.

Sun, J., Xing, Z., Xu, X., Zhu, L., Lu, Q., 2022. Heterogeneous vulnerability report traceability recovery by vulnerability aspect matching. In: IEEE International Conference on Software Maintenance and Evolution. ICSME 2022, Limassol, Cyprus, October 3-7, 2022, IEEE, pp. 175–186.

Wang, Q., Ma, Y., Zhao, K., Tian, Y., 2020. A comprehensive survey of loss functions in machine learning. Ann. Data Sci. 1–26.

Zhang, S., Ou, X., Caragea, D., 2015. Predicting cyber risks through national vulnerability database. Inf. Secur. J.: A Glob. Perspect. 24 (4–6), 194–206.